

Python Note

Chapter 1: Introduction to Python

Introduction to Python: Python is a high-level, general-purpose, object-oriented, interpreted programming language. It was created by **Guido van Rossum** and first released in 1991. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

History of Python:

1. **Genesis (Late 1980s):** Guido van Rossum, a Dutch programmer, started working on Python in the late 1980s at the Centrum Wiskunde & Informatica (CWI) in the Netherlands. He aimed to create a language that was easy to read and write, inspired by the ABC language but addressing its limitations.
2. **First Release (1991):** The first official version of Python, Python 0.9.0, was released in February 1991. This release already included many of the features that are characteristic of Python today, such as exception handling, functions, and modules.
3. **Python 1.0 (1994):** Python 1.0 was released in January 1994. This version included new features like lambda, map, filter, and reduce. The language continued to gain popularity due to its simplicity and readability.
4. **Python 2.0 (2000):** Python 2.0, released in October 2000, introduced list comprehensions, garbage collection, and Unicode support. Python 2.x series continued to be developed and improved for many years.
5. **Python 3.0 (2008):** Python 3.0, also known as Python 3000 or simply Py3k. It was released in December 2008 and aimed at fixing and enhancing various aspects of the language. This version included backward-incompatible changes to address design flaws and improve the consistency of the language.
6. **Coexistence of Python 2 and 3 (2008–2020):** The transition from Python 2 to Python 3 was gradual due to backward compatibility issues. For many years, both Python 2 and 3 versions were actively used in the development community.
7. **End of Python 2 Support (2020):** The official support of Python 2 ended on January 1, 2020. This marked the conclusion of updates, including security updates, for Python 2. The community was encouraged to migrate to Python 3.

8. **Python 3.x (ongoing):** Python 3 continues to be the focus of development, with regular releases bringing new features, optimizations, and improvements. The Python Software Foundation (PSF) oversees the development and maintenance of the language.

Features of Python:

Python is commonly used for developing websites and software, task automation, data analysis, and data visualization.

1. **Open Source:** Python is an open-source language, which means developers can view, modify, and distribute the source code of Python, and Python is also freely available on the official website.
2. **Cross-Platform:** Python is platform-independent, which means that Python code can run on various operating systems with minimum or no modification.
3. **High-Level Language:** Python is a high-level programming language because programmers don't need to remember the system architecture. This helps developers to focus on solving problems rather than dealing with complex system-specific intricacies.
4. **Object-Oriented Programming (OOP):** Python supports object-oriented programming, allowing developers to use classes and objects for organizing and structuring code.
5. **Dynamically Typed:** Python is a dynamically typed language. We don't need to specify the type of a variable at the time of declaring it. For example, if you have to assign an integer value 50 to a variable "num", you don't need to write `int num = 50`. You just have to write `num = 50`.
6. **Easy to read:** Python's syntax is straightforward and looks like simple English words. There is no use of semicolons or brackets, and the indentations define the code block.
7. **Interpreted:** Python is an interpreted language, which means that the source code is executed line by line, and not all at once.
8. **Extensive Standard Library:** Python has a wide range of libraries. This means that programmers don't have to write their code for every single thing, unlike other programming languages.

The installation process for Python:

- Go to the python official website (python.org.)
- Click on the "Downloads" tab. And download the latest version.
- After completing run the installer. Make sure to check the box that says "Add Python x.x to PATH" during installation. This makes it easier to run Python from the command prompt.
- Type "python --version" at the command prompt or power shell to check the version.

Chapter 2: Fundamentals of Python

Constant and Variables:

- A constant is a value that does not change during the execution of a program.
- Constants are often declared with the const keyword in many programming languages.
- Python does not contain any constants. Instead, Python provides us with a Capitalized naming convention method.
- Any variable written in the Upper case is considered as a Constant in Python.
- Python variables are simply containers for storing data values.
- Python is completely object-oriented, and not "statically typed". You do not need to declare their type. Every variable in Python is an object

Some example of constant: AGE= 20, NAME="Hari"

Some examples of variables: name="gita", number=20, etc.

Rules to be followed while declaring a variable:

1. A Variable's name cannot begin with a number. Either an alphabet or the underscore character should be used as the first character.
2. A variable name can only contain alpha-numeric characters and underscores (A-Z, a-z, 0-9, and _)
3. Variable names are case-sensitive (name, Name, and NAME are three different variables)
4. Variable names cannot contain reserved terms.
5. The convention in Python is to use snake_case for variable names. This means that words are lowercase and separated by underscores. For example: my_name, my_age.

Data types in Python:

A data type is an attribute associated with a piece of data that tells a computer system how to interpret its value. Understanding data types ensures that data is collected in the preferred format and that the value of each property is as expected.

1, Integer(int):

It is the most common numeric data type used to store numbers without a fractional component (-40, 0,500, 4, 2, etc.).

Example my_int = 40;

2, Floating Point (float):

It is also a numeric data type used to store numbers that may have a fractional component like monetary values do(20.04, 30.55, 0.08).

Example `float_num = 20.08;`

3, Complex Numbers:

A complex number is represented by a complex class. It is specified as (real part) + (imaginary part)`j`.

Example `complex_number = 2+3j;`

4, Strings:

- String is a text data type. You can delimit string values with either single quotes (' ') or double quotes (" "). All the characters between the opening and closing delimiter are part of the string.
- In Python, individual characters of a String can be accessed by using the method of Indexing. Negative Indexing allows negative address references to access characters from the back of the String, e.g. -1 refers to the last character, -2 refers to the second last character, and so on.
- Syntax of indexing: `variable_name[start_index:end_index:step_size]`

Example: `name = "my name is ram"`

5, Boolean (bool):

It represents the values of true and false. When working with the Boolean data type, it is helpful to keep in mind that sometimes a Boolean value is also represented as 0 (for false) and 1 (for true).

Example: `success = True`

Operators in Python: Generally operators are used to perform operations on values and variables.

Types of Python Operators:

1. **Arithmetic Operators:** Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication, and division.

<u>Name</u>	<u>Operator</u>	<u>Example</u>
Addition	+	$a=5, b=10$, e.g. $a+b=15$
Subtraction	-	$a=15, b=10$, e.g. $a-b=5$
Multiplication	*	$a=5, b=10$, e.g. $a*b=50$
Division	/	$a=4, b=2$, e.g. $a/b=2$
Exponentiation	**	$a=4, b=2$, e.g. $a**b=16$
Modulus	%	$a=5, b=2$, e.g. $a\%b=1$

2. **Assignment Operators:** Assignment operators are used to assign values to variables.

<u>Name</u>	<u>Operator</u>	<u>Example</u>
Assignment Operator	=	$a=20$
Addition Assignment	+=	$a+=1$ means $a=a+1$
Subtraction Assignment	-=	$a-=10$ means $a=a-10$
Multiplication Assignment	*=	$a*=2$ means $a=a*2$
Division Assignment	/=	$a-=1$ means $a=a-1$
Remainder Assignment	%=	$a\%=6$ means $a=a\%6$
Exponent Assignment	**=	$a**=3$ means $a=a**3$

3. **Comparison Operators:** Comparison operators are used to compare two values or variables and they return a Boolean result.

<u>Name</u>	<u>Operator</u>	<u>Example</u>
Is equal to	==	$8==6$ It gives False
Not equal to	!=	$8!=6$ it gives True
Greater than	>	$20>10$ it gives True
Less than	<	$20<10$ it gives a False
Greater than or equal to	>=	$10>=10$ it gives True
less than or equal to	<=	$20<=10$ it gives a False

4. **Logical Operators:** Logical operators are used to check whether an expression is True or False. They are used in decision-making.

<u>Operator</u>	<u>Description</u>	<u>Example</u>
And	True only if both the operands are True.	a and b

Or	True if at least one of the operands is True.	a or b
Not	True if the operand is False and vice-versa.	not a

5. **Bitwise Operators:** Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Operator	Meaning	Example
&	Bitwise AND	x & y = 0 (0000 0000)
	Bitwise OR	x y = 14 (0000 1110)
~	Bitwise NOT	~x = -11 (1111 0101)
^	Bitwise XOR	x ^ y = 14 (0000 1110)
Operator	Meaning	Example
>>	Bitwise right shift	x >> 2 = 2 (0000 0010)
<<	Bitwise left shift	x << 2 = 40 (0010 1000)

6. **Identity Operators:** *is* and *is not* are the identity operators, both are used to check if two values are located on the same part of the memory. Two equal variables do not imply that they are identical.

Operator	Meaning	Example
<i>is</i>	True if the operands are identical (refer to the same object)	x is True
<i>is not</i>	True if the operands are not identical (do not refer to the same object)	x is not True

7. **Membership operators:** *in* and *not in* are the membership operators. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set, and dictionary).

Operator	Meaning	Example
<i>in</i>	True if value/variable is found in the sequence	4 in x
<i>not in</i>	True if value/variable is not found in the sequence	4 not in x

Comments, Indentation, and Statements:

- Comments are non-executable text in your code that is used to explain or document the code. In Python, comments start with the hash symbol #. Comments provide useful information in your code. Comments are completely ignored by the Python interpreter during runtime. They exist solely for human readability and documentation purposes.
- Indentation refers to the spaces or tabs from the beginning of a code line. Python uses indentation to indicate a block of code. Indentation in blocks or functions is used to group them and to identify the block of code that is being executed.
- Statement is an instruction that the Python interpreter can execute. There are different types of statements in Python language as Assignment statements, Conditional statements, looping statements, etc. Statements are the executable commands that make up your program.

Input statement, output statement:

- Input statements are used to get data or information from the user during program execution. You can take user information using input() function in Python. By default, the input() function accepts only string arguments.
Example: name = input("Enter your name:")
- Python provides the print() function to display information or results to output devices.
Example print("Hello World")

Debugging in python:

- Debugging is the process of identifying, analyzing, and removing errors in your code.

Some useful ways to debug:

1. **Print Statements:** One of the most common ways to debug Python code is using the print() function. By adding print statements to your code, you can identify the value of variables at different points in the program, which can help you identify where the error is occurring but this method is time consuming for large programs.
2. **pdb (Python Debugger):** The built-in pdb library in Python is a command-line debugging tool that provides a set of commands for stepping through your code, inspecting variables, and controlling the execution of your program. You can set breakpoints in your code by using pdb.set_trace().

Example

```
Import pdb
name="ram"
pdb.set_trace()
print(name)
```

Chapter 3: Conditional Statements and Branching

Conditional statements are an essential part of programming in Python. They allow you to make decisions in your code based on certain conditions. For example, in the real world, people dance when there is music. The presence of music is a condition and when it is true, people will dance. Similarly, in Python programming, we use conditions to determine which set of instructions to execute.

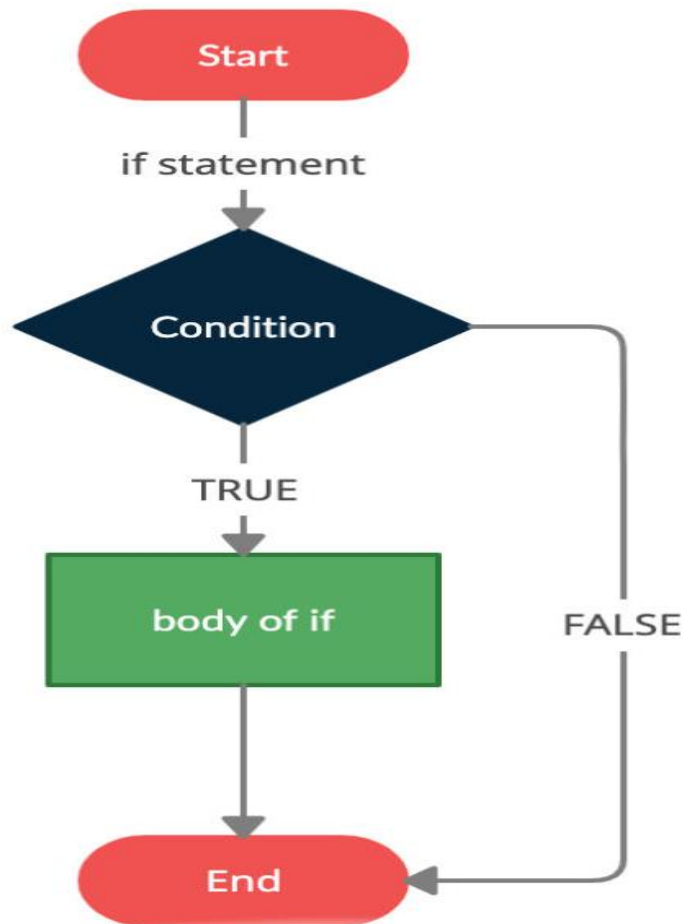
1. **If statement:** **If statement** allows you to execute a block of code if a certain condition is true. It executes a block of code only if the specified condition is true.

Syntax:

If condition:

 #code to execute **if condition** is true

Flowchart:



Example:

```
num = 10
```

```
if num>5:
```

```
    print(f"{num} is greater than 5.")
```


2. **if-else statement:** The **else statement** allows you to execute a different block of code if the **if condition** is False.

syntax:

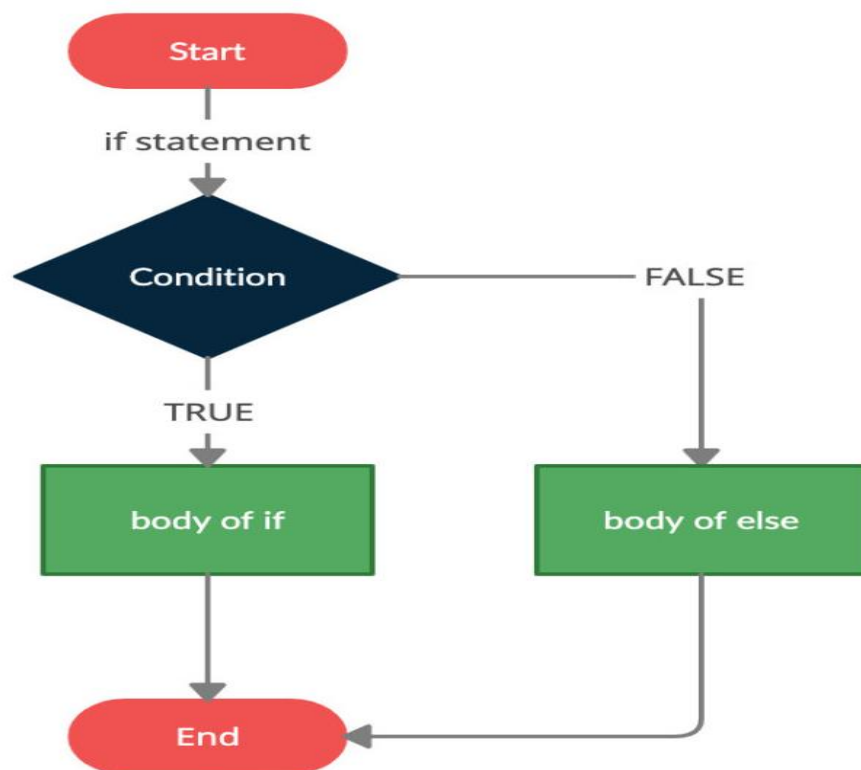
If condition:

 #code to execute **if condition** is true

else:

 #code to execute **if condition** is false.

Flowchart:



Example:

```
num = 10
```

```
if num>0:
```

```
    print(f"{num} is positive number.")
```

```
else:
```

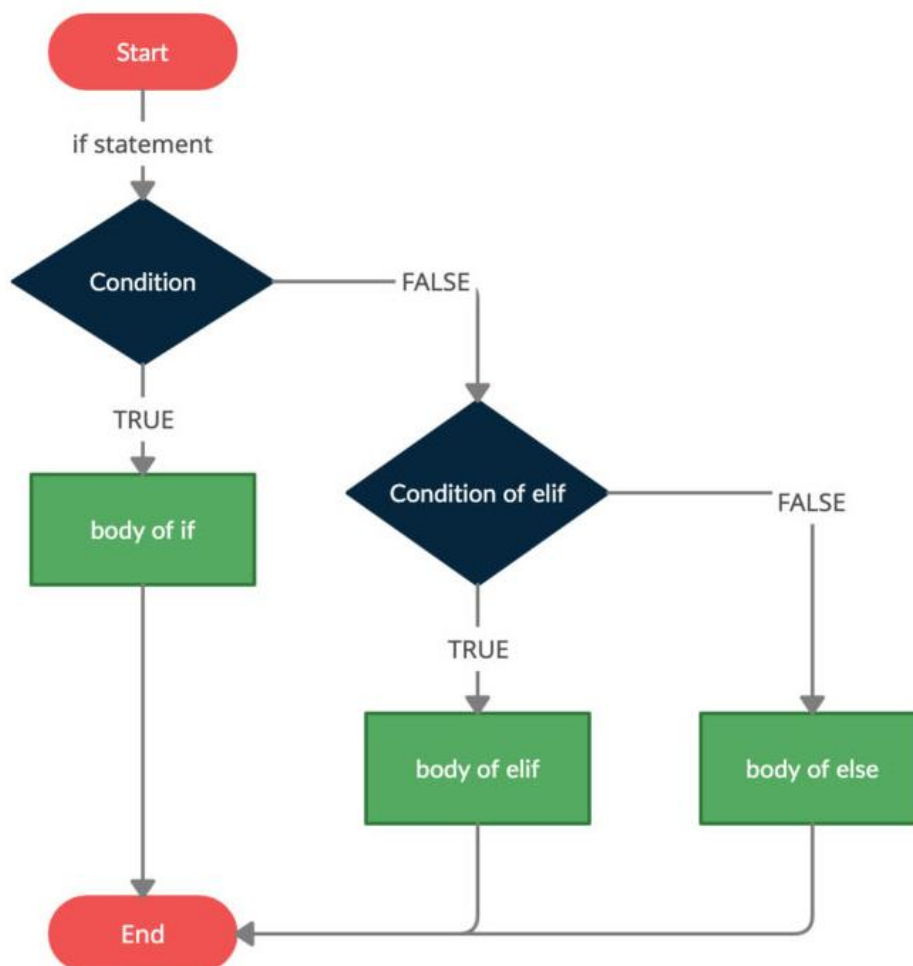
```
    print(f"{num} is negative number.")
```

3. **if-elif-else statement:** The **elif statement** allows you to check multiple conditions in sequence, and execute different code blocks depending on which condition is true.

Syntax:

```
_if condition1:  
    # code to execute if condition1 is true  
elif condition2:  
    # code to execute if condition1 is false and condition2 is true  
elif condition3:  
    # code to execute if condition1 and condition2 are false, and condition3 is true  
else:  
    # code to execute if all conditions are false
```

Flowchart:



Example:

```
marks = int(input('Enter your mark: '))  
if (marks<=20 and marks>0):  
    print('Student Failed')  
elif (marks>20 and marks<=40):  
    print('Student passed with C Grade')  
elif (marks>40 and marks<=60):  
    print('Student passed with B Grade')  
elif (marks>60 and marks<=100):  
    print('Student passed with A Grade')  
else:  
    print('Invalid marks.')
```

4. **Nested If else statement:** you can use nested if-else statements to handle more complex branching conditions

Example:

```
x = 10  
y = 5  
if x > 5:  
    print("x is greater than 5")  
    if y > 0:  
        print("y is positive")  
    else:  
        print("y is non-positive")  
else:  
    print("x is not greater than 5")  
    if y > 0:  
        print("y is positive")  
    else:  
        print("y is non-positive")
```

Chapter 4: Control Statements

Control statements are used to manage the flow of execution of a program based on certain conditions. Control statements are a powerful tool that helps to manage the flow of execution. By using control statements effectively, developers can write more efficient and effective code.

There are three types of control statements in Python.

1. **Break Statement:** The **break statement** is used to terminate a loop, i.e., for loop, while loop, or nested loop, and brings the control out of the loop. It is used with both loops(**for**, **while**), especially with nested loops (loop within a loop) to quit the loop. It terminates the inner loop and control shifts to the statement in the outer loop.

Syntax:

for variable in sequence:

 # Code inside the loop

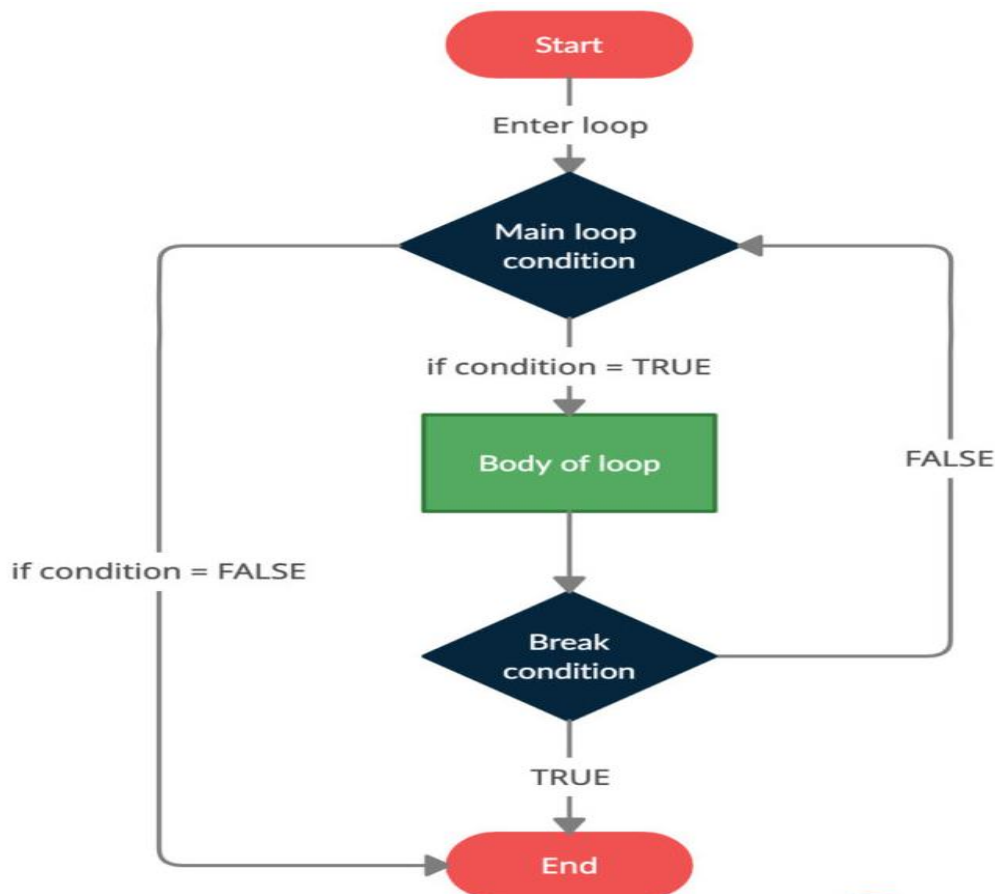
 if condition:

 break # Exit the loop if the condition is true

 # More code inside the loop

Code after the loop

Flowchart:



Example:

```
num = [10, 20, 30, 40, 50, 80]
```

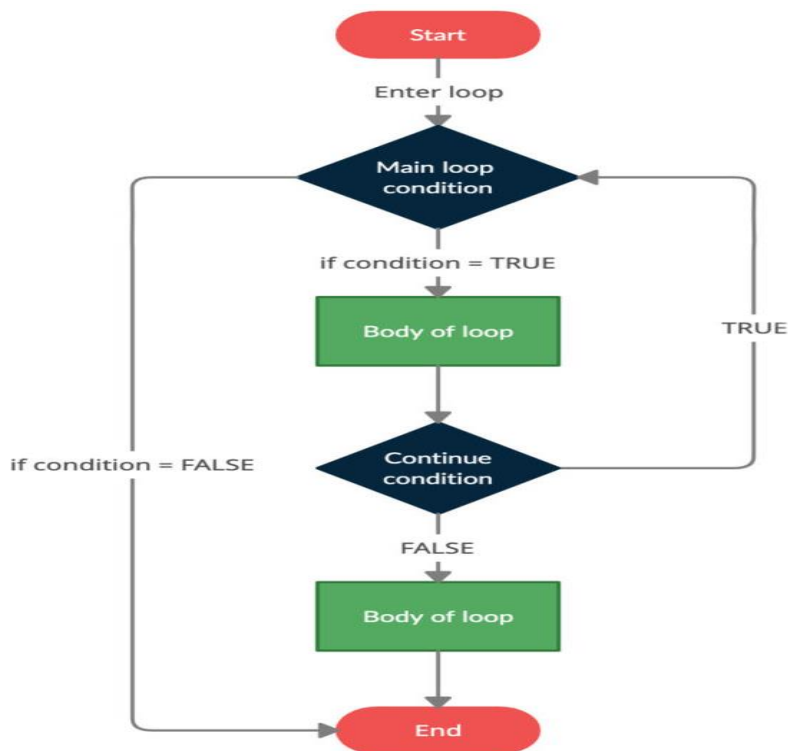
```
for i in num:
    if i == 50:
        print(f'The loop has been break with value{i}')
        break
    else:
        print(f'running loop.')
print('Outside the for loop.')
```

2. **Continue Statement:** The **continue statement** is used to skip a particular iteration of a loop when a particular condition is met. When a **continue statement** is executed inside a loop, it skips the current iteration of the loop and move to the next iteration.

Syntax:

```
for variable in sequence:
    # Code inside the loop
    if condition:
        continue # Skip the rest of the loop for the current iteration
    # More code inside the loop
# Code outside the for loop
```

Flowchart:



Example:

```
for i in range(5):  
    if i == 3:  
        continue # Skip the rest of the loop for i=3  
    else:  
        print(i)
```

3. **pass statement:** pass statements are use to write empty loops. Pass is also used for empty control statements, functions, and classes.

Example:

```
num = 20  
  
if num>30:  
    pass  
  
else:  
    print(num)
```

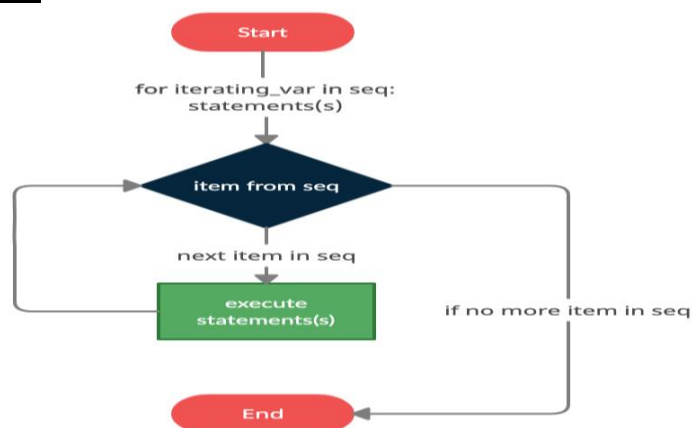
Loop in Python: In programming language, loops are used to repeat a block of code. In Python, you can use different types of loops to iterate over a sequence of elements, perform a certain operation repeatedly, or execute a block of code until a specified condition is met.

1. **for loop:** The **for** loop is used to iterate over a sequence such as a list, tuple, string, or range and execute a block of code for each item in the sequence.

Syntax:

```
for iterator_variable in sequence:  
    #Statements inside for loop for each iterator
```

Flowchart:



Example:

```
languages = ['python', 'C', 'C++', 'JavaScript']
```

```
# run a loop for each item of the list
```

```
for language in languages:
```

```
    print(language)
```

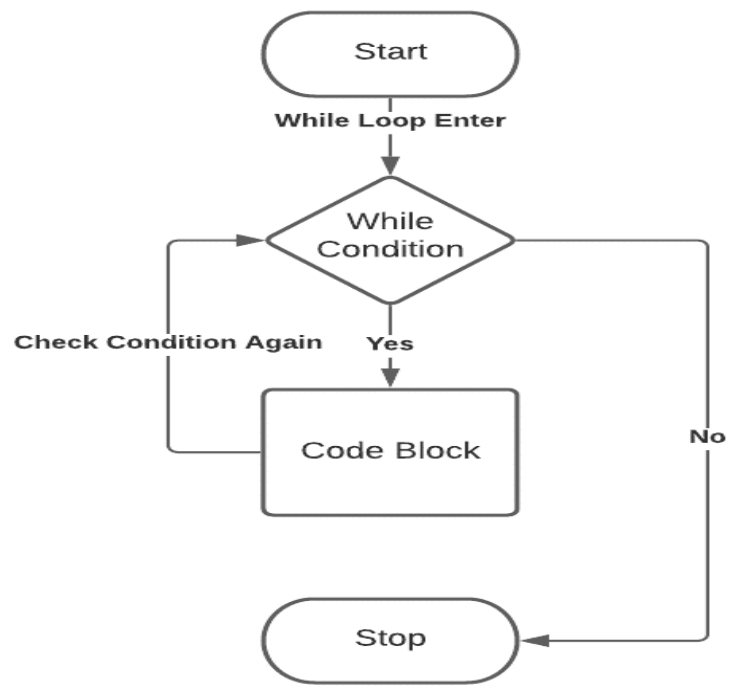
2. **while loop:** In Python, **while** loop is used to run a block of code until a certain condition is met.

Syntax:

```
while condition:
```

```
    # code to be executed while the condition is true
```

Flowchart:



Example:

```
i = 0 # initialize value of i with 0
```

```
while i < 5:
```

```
    print(i)
```

```
    i += 1 #value of i will be increase by 1
```

different between for and while loop:

<i>for loop</i>	<i>while loop</i>
<i>For loop is used to iterate over a sequence of items.</i>	<i>While loop is used to repeatedly execute a block of statements while a condition is true.</i>
<i>For loop requires a sequence to iterate</i>	<i>While the loop requires an initial condition that is tested at the beginning of the loop.</i>
<i>Uses for keyword</i>	<i>Uses while keyword</i>
<i>The for loop is faster than the while loop.</i>	<i>While loop is relatively slower as compared to for loop.</i>
<i>In For loop, Range or xrange function is used to iterate.</i>	<i>No such function is used in the while loop.</i>
<i>Generally, For loop is used when the number of iterations is already known.</i>	<i>Generally, While loop is used when the number of iterations is already Unknown.</i>
<i>In absence of condition, Loop runs infinite times.</i>	<i>In absence of condition, Display the compile time error.</i>

Nested Loop: In Python, a nested loop is a loop inside another loop. You can have a **for loop** inside another **for loop**, a **while loop** inside another **while loop**, or a combination of both. Nested loops are useful when you need to perform repetitive tasks within repetitive tasks.

Nested For loop example:

```
for i in range(2):  
    for j in range(2):  
        print(i, j)
```

Nested while loop example:

```
i = 0  
while i <= 2:  
    j = 0  
    while j <= 2:  
        print(i, j)  
        j += 1  
    i += 1
```


Chapter 7 : functions

Defining a Function:

In programming, a function is a reusable block of code that executes a certain functionality when it is called. In python we create function using def keyword. Functions help in modularizing code, making it more organized, readable, and maintainable.

Syntax:

```
def name_of_function(parameter):# this is function definition
```

```
    #function body
```

```
name_of_function(argument) #this is function called.
```

In python, we can pass information to a function using parameters.

Example:

```
def fun1(name):
```

```
    print(f" my name is {name})
```

```
fun1("ram")
```

name is the parameter. It's a hold the actual value that will be passed when calling the function.

"ram" is the argument. It's the actual value that is passed to the function.

Note: you can have multiple parameters in a function, and you can pass values to function but one main thing is that number of argument and parameter must be match.

Example:

```
def add_number(a, b):
```

```
    print(f"The sum of {a} and {b} is {a+b}")
```

```
add_number(10, 20)
```

In above example 10 and 20 are argument and a and b are parameter a hold 10 and b hold 20.

Passing Arguments:

Positional Arguments in python: When we call a function with positional arguments, the values you provide are matched to the function parameters based on their order.

Example:

```
def add_number(a, b):  
    print(f"The sum is {a+b}")  
  
add_number(10, 20)
```

In the example: `add_number` is the function name. `(a, b)` are the parameters, and they are positional because their order matters. The first value passed by function is taken by the first parameter at function call.

10 is passed as the value for `a`, and 20 is passed as the value for `b` when calling the function.

When you call `add_number(10, 20)`, Python assigns 10 to `a` and 20 to `b` based on their positions in the function call.

Keyword Arguments: In Python, keyword arguments allow you to pass values to a function by explicitly specifying the parameter names along with their corresponding values. This way, the order in which you provide the arguments doesn't matter.

Example:

```
def add_number(a, b):  
    print(f"The sum is {a+b}")  
  
add_number(b=10, a=20)
```

Default Values: In Python, we can provide default values for function parameters. Default values are used when a value for a corresponding parameter is not provided during the function call.

Example:

```
def add_number(a, b=0):  
    print(f"The sum is {a+b}")  
  
add_number(10, 20)  
add_number(10)
```

Note: if you can not pass from function call it takes default value.

Equivalent Function Calls: we can achieved equivalent function through different combinations of positional and keyword arguments. Here's an example illustrating equivalent function calls:

Example:

```
def add_number(a, b=0):
```

```
    print(f"The sum is {a+b}")
```

```
# Equivalent function calls:
```

```
# 1. Using only positional arguments
```

```
add_number(1, 2,)
```

```
# 2. Using positional and keyword arguments
```

```
add_number(1, b=2)
```

```
# 3. Using only keyword arguments
```

```
add_number(a=1, b=2)
```

```
# 4. Using default values for some parameters
```

```
add_number(1, 2)
```

Avoiding Argument Errors: To avoid argument error in function you can pass data as keyword argument(**) and non-keyword argument(*). You can also give type hint in function definition. You can also you default value in function definition.

Syntax:

```
def fun1(normal_arg, *args, **kwargs):
```

```
    #function body
```

```
fun1('ram', 10, 30, 40, name="hari", address="itahari")
```

In above example ram value is take by normal_arg and 10, 30, 40 is value take by args as a tuple and name="hari" and address="itahari" are taken by kwargs as a dictionary.

Return Types: In Python, you can use the **return** keyword to exit a function so it goes back to where it was called. That is, send something out of the function. The **return** statement can contain an expression to execute once the function is called. The return type of a function is not explicitly declared, and Python is dynamically typed, meaning that the type of the return value is determined at runtime. If a function doesn't have a **return** statement, it implicitly returns **None**.

Example:

```
def add_number(a, b):  
    total = a+b  
    return total
```

```
result = add_number(10, 20)  
print(f"sum is {result}")
```

The result variable store the vaule return by function.

Note: you can return output from many way like list, dictionary, string, number it depends on your requirements.

Example:

```
return {"key": "value", "key1": "value1"} # it returns a Dictionary data.  
return [10, 20, 30] # it returns a list data.
```

Passing a List: you can pass a list to a function as an argument.

example:

```
def add_number(my_list):  
    my_list.append(20)  
    print(f"data inside function {my_list}")  
data = [10, 20, 30, 40]  
add_number(data)  
print(f"data outside function {data}")
```

Note: when you pass a list to a function, the function can modify the original contents of the list directly because lists are mutable objects.

Preventing a Function from Modifying a List:

If you want to prevent a function from modifying the original list passed to it, you can take a few approaches:

- You can pass copy of the original list to the function.

Example:

```
def add_number(my_list):  
    my_list.append(20)  
    print(f"data inside function {my_list}")  
  
data = [10, 20, 30, 40]  
  
add_number(data.copy())#pass copy of original list.  
  
print(f"data outside function {data}")
```

- You can assign original list to new variable inside function and you can apply any operation on new variable.

Example:

```
def add_number(my_list):  
    new_list = my_list #assign original list to new variable  
    new_list.append(20)  
    print(f"data inside function {new_list}")  
  
data = [10, 20, 30, 40]  
add_number(data)  
print(f"data outside function {data}")
```

Storing Your Functions in Modules: In Python, Modules are simply files with the ". py" extension containing Python code that can be imported inside another Python Program. It contains Python Functions, Classes or Variables.

You can import your one module inside another.

Syntax: import module_name # Import the entire module

You can import specific function, class or variable from another module.

Syntax: from module_name import function_name, class_name, variable_name # Import specific functions, classes and variables from another module.

Using as to Give a Function an Alias: In Python, you can use the **as** keyword to give a function (or a module) an alias when importing it. This can be useful when working with libraries or modules that have long or commonly used names.

Example:

```
import matplotlib.pyplot as plt # Importing a module with an alias
```

```
from math import sqrt as square_root # Importing a function with an alias
```

Bringing Everything into the Current Scope: In Python, you can use the `from module import *` syntax to bring all names defined in a module into the current scope. However, this practice is generally discouraged because it can lead to namespace pollution, making it difficult to track and understand where each name comes from.

Example:

```
# Importing all names from the math module into the current scope
```

```
from math import *
```

Chapter 8: Classes and Objects

object-oriented programming:

- Object-oriented programming is a programming paradigm based on the concept of objects, which can contain data and code: data in the form of fields, and code in the form of procedures.
- Paradigm can also be termed as a method to solve a problem or do some task.
- OOPS is not any programming language but OOPS is a different style of writing code.
- Smalltalk is a world-first pure OOPS programming language.

Class:

- Python is an object-oriented programming language.
- In Python, Almost everything is an object.
- A class is a user-defined blueprint(prototype)of objects.
- We can think of the class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows, etc. Based on these descriptions we build the house. House is the object.
- Classes provide a means of bundling data and functionality together.
- Class is a collection of objects. It is not a real-world entity.it is just a template or prototype.
- Class does not occupy memory.

Objects:

- Object is an instance of class.
- Object is a real-world entity.
- Object occupies memory.
- We can create many objects from the same class.
- Software programmers use the same class, and thus the same code, over and over again to create many objects. So objects provide the benefit of modularity and information hiding and classes provide the benefit of reusability.

Syntax:

class NameOfClass:#class name should be pascal case as per PEP8 guide line.

 //method and data of class

C1 = NameOfClass() #creating object c1 is object.

Example:

```
class Dog:
```

```
    pass
```

bull_dog = Dog()#bull_dog is an object of Dog that can access the method and attribute of the class.

Constructors:

In Python, a constructor is a special method within a class that is automatically called when an object of the class is instantiated. The purpose of a constructor is to initialize the attributes (variables) of the object with certain values or perform other setup tasks. In Python, the `__init__()` method is called the constructor and is always called when an object is created.

Syntax:

```
def __init__(self):
```

```
    # body of the constructor
```

Example:

```
class Car:
    # initializing constructor
    def __init__(self, name, color):
        self.name = name
        self.color = color
    # ceate method to show data members
    def show_details(self):
        print(f"the car name is {self.name} and color is {self.color}.")
# creating object of the Car class
obj = Car("BMW", "white")#creating class object with constructor data
initialize.when we make object the __init__() method will be automatically
called.
# calling the instance method using the object obj
obj.show_details()
```


Setting a Default Value for an Attribute:

```
class Car:
    def __init__(self, brand="Tesla", model="model Y"):
        #tesla and model Y are default attributes.
        self.brand = brand
        self.model = model
# Creating instances of the Car class without providing values
car1 = Car()
car2 = Car("BMW", "i7")
# Accessing attributes of the objects
print(car1.brand) # Output: Tesla
print(car1.model) # Output: model Y
print(car2.brand) # Output: BMW
print(car2.model) # Output: it
```

Modifying Attribute Values:

```
class Car:
    def __init__(self, brand):
        self.brand = brand
    def display_info(self):
        print(f"Car Info: {self.brand}")
# Creating an instance of the Car class
car1 = Car(brand="Toyota")
# Accessing and displaying initial attribute values
car1.display_info() # Output: Car Info: Toyota
# Modifying attribute values
car1.brand = "Honda"
# Accessing and displaying modified attribute values
car1.display_info() # Output: Car Info: Honda.
```

Inheritance:

- The process of inheriting the properties of parent class to child class is called inheritance.
- The existing class is called a base class or parent class.
- The new class is called a subclass child class or derived class.
- The main purpose of inheriting is code reusability.

- In inheritance, the child class acquires all the data members properties and methods from the parent class,

Syntax:

Class BaseClass:

 #body of the base class

Class DerivedClass(BaseClass):

 #body of the derived class

Example:

```
class Person(object):
    # Parent class Constructor
    def __init__(self):
        self.name = "Hari"
    #parent class method
    def show_parent(self):
        print("this is parent class method.")

# making Subclass
class Employee(Person):#Class Employee inherit from class Person
    # child class method
    def show_child(self):
        print("this is child class method.")

emp = Employee() # An Object of Employee
emp.show_parent()#calling parent class method by using child class object.
emp.show_child()#calling child class method by using child class object.
```

Types of inheritance:

1. Single inheritance:

A child class inherits from a single-parent class. This is the simplest form of inheritance.

Example:

```
class Vehicle:
    # Parent class Constructor
    def __init__(self):
        self.name = "Hari"
    #parent class method
    def show_parent(self):
        print("this is parent class method.")

# making Subclass
class Car(Vehicle):#Class Employee inherit from class Person
    # child class method
    def show_child(self):
        print("this is child class method.")

car = Car() # An Object of car class
car.show_parent()#calling parent class method by using child class object.
car.show_child()#calling child class method by using child class object.
```

2. Multiple Inheritance:

One child class can inherit from multiple parent classes.

Example:

```
# Base class1
class Mother:
    mother_name = "Sita"
    def mother(self):
        print(self.mother_name)
# Base class2
class Father:
    father_name = "Hari"
    def father(self):
        print(self.father_name)
# Derived class
class Son(Mother, Father):
    def show_details(self, name):
        print(f" my name is {name} father name {self.father_name} mother name is {self.mother_name}")
# creating obj of son class which inherit from Mother and Father class.
s1 = Son()
s1.show_details("ram")
```

3. Multilevel Inheritance:

In multilevel inheritance, a class can inherit from a base class, and then another class can inherit from the derived class. This creates a chain of inheritance.

Example:

```
class VehicleCompany:
    # Parent class Constructor
    def __init__(self):
        self.name = "Vehicle name"
    #parent class method
    def show_vehicle(self):
        print("this is vehicle class method.")
# making Subclass
class Car(VehicleCompany):#Class Car inherit from class VehicleCompany
    # child class method
    def show_car(self):
        print("this is child class method.")
# making Subclass
class TeslaCar(Car):#Class TeslaCar inherit from class Car
    # child class method
    def tesla_info(self):
        print(f"this is tesla car class method.and company is {self.name}")
car = TeslaCar() # An Object of car class
car.show_vehicle()
car.show_car()
car.tesla_info()
```

4. Hierarchical inheritance:

When more than one derived class is created from a single base this type of inheritance is called hierarchical inheritance.

Example:

```
# Base class
class Parent:
    def show_parent(self):
        print("This function is in parent class.")
# Derived class1
class Child1(Parent):
    def show_child1(self):
        print("This function is in child 1.")
# Derived class2
class Child2(Parent):
    def show_child3(self):
        print("This function is in child 2.")
# Driver's code
object1 = Child1()
object2 = Child2()
object1.show_parent()
object1.show_child1()

object2.show_parent()
object2.show_child3()
```

5. Hybrid Inheritance:

Inheritance consisting of multiple types of inheritance is called hybrid inheritance.

Example:

```
class School:
    def show_school(self):
        print("This function is in school.")
class Student1(School):
    def show_student1(self):
        print("This function is in student 1. ")
class Student2(School):
    def show_student3(self):
        print("This function is in student 2.")
class Student3(Student1, School):
    def show_student4(self):
        print("This function is in student 3.")
# Driver's code
object = Student3()
object.show_school()
object.show_student1()
```

Overriding Methods from the Parent Class in Python:

In Python, you can override methods from the parent class in the child class. This means that a child class can provide a specific implementation for a method that is already defined in its parent class. This is achieved by creating a method with the same name in the child class as the one in the parent class.

Example:

```
class Vehicle:
    def make_sound(self):
        print("parent class method.")

class Car(Vehicle):
    def make_sound(self):
        print("car class method.")

class Bus(Vehicle):
    def make_sound(self):
        print("bus class method.")

# Creating instances of the classes
ve = Vehicle()
car = Car()
bus = Bus()

# Calling the overridden method
ve.make_sound() # Output: parent class method.
car.make_sound() # Output: car class method.
bus.make_sound() # Output: bus class method.
```

Defining Attributes and Methods for the Child Class and super() keyword:

In Python, the super() function is used to refer to the parent class or superclass. It allows you to call methods defined in the superclass from the subclass, enabling you to extend and customize the functionality inherited from the parent class.

Example:

```
class Parent:
    def __init__(self, parent_name):
        self.parent_name = parent_name

class Child(Parent):
    def __init__(self, parent_name, name):
        self.name = name
        # call superclass
        super().__init__(parent_name)
    def show_details(self):
        print(f" my name is {self.name} and parent name is {self.parent_name}")

child = Child("Hari and Sita","ram")
child.show_details()
# Output: my name is ram and parent name is Hari and Sita
```

Introduction to Importing Classes with example:

In Python, you can organize your code into modules, and within those modules, you may have classes or functions. To use classes from one module in another module, you need to import them.

Syntax:

From file_name import ClassName

Or

From file_name import ClassName as cls #name aliasing of ClassName.

Chapter 9: FILES, DIRECTORIES AND EXCEPTIONS

Files: In Python, a file is a named location on disk or in memory used to store data. Variables, lists, dictionaries, sets, arrays, pandas series, and DataFrame offer only temporary data storage. The data is lost when a local variable “goes out of scope” or when the program terminates. Files are permanent storage areas to store your data.

File Handling in Python: file handling refers to the process of performing operations on files, such as creating, reading, writing, and closing them. Python provides a built-in file object that you can use to perform these operations.

Advantages of file:

- Stored data is permanent unless someone removes it.
- Stored data can be shared.
- It is possible to update or remove the data.

Types of files in Python: there are two main types of files based on the data they store.

1. **Text file:** it stores data in the form of characters or plain text format. it is used to store characters and strings. Text files are used for storing human-readable data. Some common extensions are ‘.txt’, ‘.html’, ‘.csv’ etc.
2. **Binary file:** it stores data in the form of bytes a group of 8 bits. The content may not be easily human-readable. It is used to store non-textual data, such as images, videos, audio, executable programs, or any data that doesn't follow a plain text format. Some common extensions are ‘.jpg’, ‘.png’, ‘.exe’, ‘.pdf’ etc.

Opening and closing a file: In Python open() function is used to open a file and close() method is used to close a file.

Syntax:

File_object = open(“file_name”, “mode”) #open file in specified mode

#read write append operation

file_object.close() #opening file will be close.

File_name: The name of the file you want to open.

mode: The mode in which the file should be opened (e.g., read, write, append,).

Example:

```
f=open("student.txt", mode="w")#student file will be open in write mode.
f.write("my name is ram.")#write in this file
f.close()#file will be close
```


Reading and Writing from a file:

To read a file, first open an existing file with an extension in read mode then read the entire contents of the file into the variable content using the read() method and print it. If not found file it gives FileNotFoundError.

Example:

```
# Open the file in read mode
with open('filename.txt', 'r') as file:
    # Read the entire contents of the file
    file_content = file.read()
    print(file_content)
```

To write inside the file, the file must be opened with a file name and extension if a file already exists it overwrites the file content else it creates a new file.

```
Example: # Open the file in write mode
with open('filename.txt', 'w') as file:
    # Write content to the file.
    file.write('content to be written to the file!')
```

file modes:

Mode	Description
w	- This mode is used to open an existing file for write operation. - If the file contains some data already, then the new data will override the old one. - If the specified file is not already available, then this mode will create that file.
r	- This mode is used to open an existing file for read operation. - After opening, the file pointer is positioned at the beginning of the file. - If the specified file does not exist, then we will get FileNotFoundError. - If we don't mention the mode while opening a file then by default python will open it in this mode.
a	- This mode is used to open an existing file for append operation. - If the specified file doesn't exist, then this mode will create a new file. - It won't override the existing data.
w++	- To write and read data from a file. - The previous data, if any, will be overridden.
r++	- To read and write data to and from the file. - The previous data, if any, will be overridden. - After opening, the file pointer is positioned at the beginning of the file.
a++	- To append and read data from the file. - File pointer is end of the file if the file exists, - A new file is created for reading and writing, if the specified file doesn't exist.
x	- To open the file in exclusive creation mode - The file creation fails if the file already exists

Reading line by line: Python offers several methods to read files line by line, including the 'readline' method, the 'readlines' method, and the 'for' loop method. Each method has its unique advantages and use cases.

Method	Description	Ideal Use-Case
'readline'	Simple, reads one line at a time, requires a while loop and manual line reading	When simplicity is preferred
'readlines'	Convenient, returns a list of lines, reads entire file into memory	When all lines need to be stored in a list for further processing
'for' loop	Efficient, suitable for large files, reads one line at a time	When dealing with large files

The 'readline' Method: The 'readline' method is a simple, and effective way to read a file line by line in Python. When invoked, it reads the subsequent line from the file and returns it as a string. Example:

```
with open('file_name.txt', 'r') as f:
    line = f.readline()
    while line:
        print(line)
        line = f.readline()
```

The 'readlines' Method: The 'readlines' method, alternatively, reads all lines from the file simultaneously and returns them as a list of strings. Each string in the list represents a line in the file. Example:

```
with open('file_name.txt', 'r') as f:
    lines = f.readlines()
    for line in lines:
        print(line)
```

The 'for' Loop Method: you can be used a 'for' loop to iterate over the file object, which will return one line at a time. This method is efficient and doesn't load the entire file into memory, making it suitable for large files. example:

```
with open('file_name.txt', 'r') as f:
    for line in f:
        print(line)
```

Handling big Files in Python: When dealing with large files, reading the entire file into memory can be inefficient or even impossible if the file is larger than the available memory. In such cases, you can read the file line by line or in chunks. You can read a large file in chunks:

```
with open('student.txt', 'r') as file:
    while True:
        chunk = file.read(1024)
        if not chunk:
            break
```

In this example, we're reading the file in chunks of 1024 bytes and processing each chunk separately. This method allows you to handle large files efficiently without loading the entire file into memory.

Renaming, Moving, Copying, and Removing Files: In Python, you can perform various file operations like renaming, moving, copying, and removing files using functions from the `os` and `shutil` modules.

Renaming a File:

```
import os
# Rename a file
os.rename('old_filename.txt', 'new_filename.txt')
```

Moving a File:

```
import shutil
# Move a file to a new location
shutil.move('old_location/old_filename.txt', 'new_location/new_filename.txt')
```

Copying a File:

```
import shutil
# Copy a file to a new location
shutil.copyfile('source_filename.txt', 'destination_filename.txt')
```

Removing a File:

```
import os
# Remove a file
os.remove('filename_to_remove.txt')
```

Error Handling and Exceptions in File Handling: Error handling is an important aspect of file handling. When working with files, many things can go wrong. The file might not exist, you might not have permission to access it, or there could be an error while reading or writing the file. Example

```
try:
    with open('myfile.txt', 'r') as file:
        for line in file:
            print(line)
except FileNotFoundError:
    print('File not found')
except IOError:
    print('An error occurred while reading the file')
```

Exception Handling in Python: Exception handling in Python allows you to handle errors and unexpected behavior in your code. It helps prevent your program from crashing by providing mechanisms to catch and respond to exceptions. Python provides a try, except, else, and finally blocks for handling exceptions.

Syntax:

```
try :
    #statements in try block
except :
    #executed when error in try block
```

The try statement works as follows.

- First, the try clause (the statement(s) between the try and except keywords) is executed.
- If no exception occurs, the except clause is skipped and execution of the try statement is finished.
- If an exception occurs during execution of the try clause, the rest of the clause is skipped. Then, if its type matches the exception named after the except keyword, the except clause is executed, and then execution continues after the try/except block.
- If an exception occurs which does not match the exception named in the except clause, it is passed on to outer try statements; if no handler is found, it is an unhandled exception and execution stops with an error message.

Example:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    # Handle specific exception (division by zero)
    print("Error: Division by zero!")
except FileNotFoundError:
    # Handle specific exception (file not found)
    print("Error: File not found!")
except Exception as e:
    # Handle any other exceptions
    print("An error occurred:", e)
else:
    # This block executes if no exceptions are raised in the try block
    print("No exceptions occurred!")
finally:
    # This block always executes regardless of whether an exception occurred or not
    # It's typically used for cleanup operations, like closing files or releasing resources
    print("Cleaning up...")
```

Raise Exception: In Python, you can raise exceptions using the raise statement. This allows you to manually trigger an exception under certain conditions.

Example:

```
try:
    age = int(input("Enter your age:"))
    if age<=16:
        raise Exception("you can not vote")
    print("you can vote")
except Exception as exe:
    print(exe)
```

We raise an Exception with a specific error message if the value of age is less than and equal to 16.

Chapter 10: Machine Learning/Deep Learning Framework

NumPy:

- Numpy is a scientific computing library for Python.
- It supports a large number of data in the form of multi-dimensional array and matrix.
- It is a Python library that provides a multidimensional array object, and various derived objects (such as masked arrays and matrices).
- It is used for mathematical calculations like linear algebra, Fourier transform, etc.
- It is widely used in scientific computing, data analysis, and machine learning applications due to its speed and versatility.
- It also includes array-aware functions for creating, reshaping, concatenating, and padding arrays; searching, sorting, and counting data; and reading and writing files.

Syntax:

Import numpy as np

Example:

```
arr1 = np.array([1,2,3,4])# one dimensional array
```

```
arr2= np.array([[1,2,3,4],[10,20,30,40]]) #two dimensional array
```

pandas:

- Pandas is a Python library.
- Pandas is used to analyze data.
- Pandas is built on top of the NumPy library and is particularly well-suited for working with tabular data, such as spreadsheets or SQL tables.
- Pandas is a powerful and versatile library for data manipulation and analysis in Python, making it a favorite tool among data scientists, analysts, and researchers for working with structured data.
- It offers a rich set of functions to import and process various types of file formats from multiple data sources. The Pandas library is specifically useful for data scientists working with data cleaning and analysis.

Pandas read 2 types of data structures:

1. **Series**: A one-dimensional labeled array capable of holding any data type (integers, strings, floating-point numbers, Python objects, etc.). It can be thought of as a single column from a data frame or as a one-dimensional array with an index.
2. **DataFrame**: A two-dimensional labeled data structure with columns of potentially different data types. It can be thought of as a table or spreadsheet, where each column represents a different variable, and each row represents a different observation or sample.

Series vs DataFrame

Series	DataFrame
One-dimensional	two-dimensional
Homogenous – Series elements must be of the same data type.	Heterogenous – DataFrame elements can have different data types.
Size-immutable – Once created, the size of a Series object cannot be changed.	Size-mutable – Elements can be dropped or added in an existing DataFrame.

Syntax:

Import pandas as pd

Example of series:

```
pd.Series(['hari','shyam', 'gita', 'ram'])#to create series data structure
```

Example of DataFrame:

```
l1= [['ram', 10, 'kathmandu'], ['hari',20,'pokhara'], ['rita',30, 'dharan']]
```

```
data = pd.DataFrame(l1, columns=['name','age', 'address']) #to create DataFrame from list
```

Scikit-learn:

Scikit Learn (aka sk-learn) is a library with the implementation of several machine learning and pre-processing algorithms which was released by **David Cournapeau** in 2007. It features various

classification, regression, and clustering algorithms including support vector machines, random forests, gradient boosting, k-means, and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries Numpy and Scipy.

In order to use scikit-learn and to apply an algorithm, we need to follow some guidelines and steps that are required to be performed in order. We cannot start the training of a model on an ineligible dataset. Here are they:

- Read the data in a DataFrame.
- Prepare the data or apply pre-processing techniques.
- Decide what algorithm is best for what type of data and this is not something easy to learn at this point, for this we will need to study details about different models, their pros and cons, what are hyperparameters, etc.
- Choose a class of models by importing the appropriate estimator class from scikit-learn.
- Choose model hyperparameters by instantiating this class with desired values.
- Arrange data into a features matrix and target vector following the discussion above.
- Fit the model to your data by calling the fit() method of the model instance.
- Apply the Model to new data:
 - For supervised learning, often we predict labels for unknown data using the predict() method.
 - For unsupervised learning, we often transform or infer properties of the data using the transform() or predict() method.

Keras: Keras is a deep learning library written in Python and capable of running on top of either JAX, TensorFlow, or PyTorch. It was developed by one of the Google engineers, **Francois Chollet**. It is made user-friendly, extensible, and modular for facilitating faster experimentation with deep neural networks. Which is used to build machine learning models. Keras is based on a minimal structure that provides a clean and easy way to create deep learning models based on TensorFlow or Theano. Keras is designed to quickly define deep learning models. Well, Keras is an optimal choice for deep learning applications.

Features of Keras:

- Consistent, simple and extensible API.
- Minimal structure - easy to achieve the result without any frills.
- It supports multiple platforms and backends.
- It is a user-friendly framework that runs on both CPU and GPU.

Benefits of Keras:

- Larger community support.

- Easy to test.
- Keras neural networks are written in Python which makes things simpler.
- Keras supports both convolution and recurrent networks.

5 steps of the Keras life cycle:

1. Define Network:

In this step, you define the architecture of your neural network model using Keras API. This involves deciding the number of layers, types of layers, activation functions, and connections between them.

Example:

```
model = Sequential()
model.add(Dense(2))
```

- #### **2. Compile Network:**
- After creating the model, you need to compile it before training. Compiling involves specifying the optimizer, loss function, and metrics that will be used to evaluate the model's performance.

```
Example: model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

- #### **3. Fit Network:**
- This step involves feeding training data into the model and adjusting the weights of the model's neurons through backpropagation in order to minimize the defined loss function.

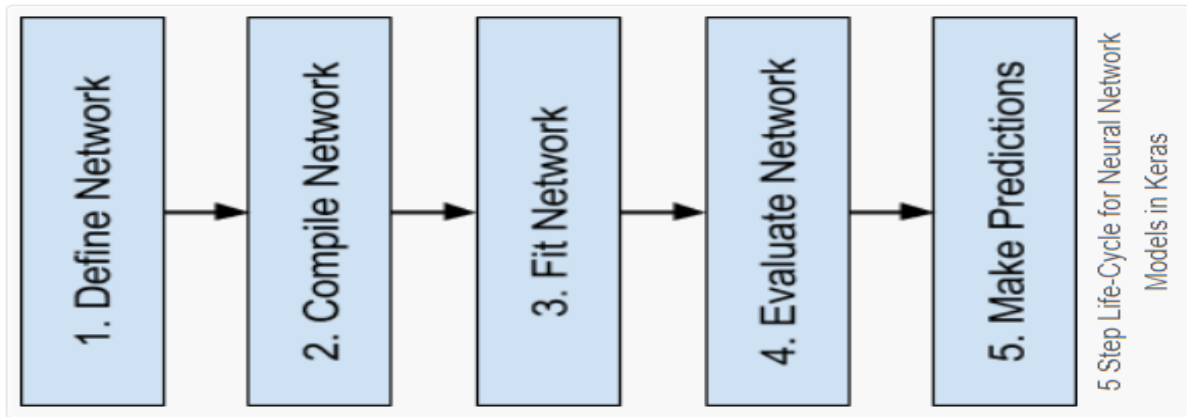
```
Example: model.fit(x_train, y_train, epochs=10, batch_size=32,
validation_data=(x_val, y_val))
```

- #### **4. Evaluate Network:**
- After training the model, you evaluate its performance on a separate dataset (usually a validation set or a test set) to assess how well it generalizes to unseen data.

```
Example: test_loss, test_accuracy = model.evaluate(x_test, y_test)
```

- #### **5. Make Predictions:**
- Once the model is trained and evaluated, it can be deployed for making predictions on new, unseen data. This step often involves integrating the model into an application or system where it can be used to make real-world predictions.

```
Example: predictions = model.predict(x_new_data)
```



Example:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
# Load and preprocess the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train / 255.0 # Normalize pixel values to range [0, 1]
x_test = x_test / 255.0
y_train = to_categorical(y_train, 10) # One-hot encode labels
y_test = to_categorical(y_test, 10)

# Define the model architecture
model = Sequential([
    Flatten(input_shape=(28, 28)), # Flatten 28x28 images to a 1D vector
    Dense(128, activation='relu'), # Dense layer with 128 neurons, ReLU
    activation
    Dense(10, activation='softmax') # Output layer with 10 neurons (for 10
classes), softmax activation
])
# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=5, batch_size=32, validation_data=(x_test,
y_test))
# Evaluate the model on test data
test_loss, test_acc = model.evaluate(x_test, y_test)
```

```
print(f'Test accuracy: {test_acc}')
```

TensorFlow: TensorFlow is an open-source machine learning framework developed by Google Brain Team and released in 2015. It is one of the most popular and widely used libraries for building and training machine learning and deep learning models. TensorFlow provides a comprehensive ecosystem of tools, libraries, and community resources to support various machine-learning tasks. TensorFlow allows developers to create dataflow graphs—structures that describe how data moves through a graph, or a series of processing nodes. Each node in the graph represents a mathematical operation, and each connection or edge between nodes is a multidimensional data array, or tensor.

TensorFlow with Python: Many programmers access TensorFlow by way of the Python programming language. Python is easy to learn and work with, and it provides convenient ways to express and couple high-level abstractions. TensorFlow is supported on Python versions 3.7 through 3.11, and while it may work on earlier versions of Python it's not guaranteed to do so.

Nodes and tensors in TensorFlow are Python objects, and TensorFlow applications are themselves Python applications. The actual math operations, however, are not performed in Python. The libraries of transformations that are available through TensorFlow are written as high-performance C++ binaries. Python just directs traffic between the pieces and provides the programming abstractions to hook them together.

High-level work in TensorFlow—creating nodes and layers and linking them together—relies on the Keras library. The Keras API is outwardly simple; you can define a basic model with three layers in less than 10 lines of code, and the training code for the same takes just a few more lines. But if you want to "lift the hood" and do more fine-grained work, such as writing your own training loop, you can do that.

Example:

```
# importing tensorflow
import tensorflow as tf
```

```

# creating nodes in computation graph
node1 = tf.constant(3, dtype=tf.int32)
node2 = tf.constant(5, dtype=tf.int32)
node3 = tf.add(node1, node2)

# create tensorflow session object
sess = tf.compat.v1.Session()

# evaluating node3 and printing the result
print("sum of node1 and node2 is :",sess.run(node3))
# closing the session
sess.close()

```

different between Keras and TensorFlow:

<u>Keras</u>	<u>TensorFlow</u>
Keras is a high-level API that is running on top of TensorFlow, CNTK, and Theano.	TensorFlow is a framework that offers both high and low-level APIs.
Keras is easy to use if you know the Python language.	You need to learn the syntax of using various TensorFlow function
It can be used for low-performance models.	It is used for high-performance models.
Keras is usually used for small datasets	TensorFlow is used for high-performance models and large datasets.
Keras has a simple architecture and easy to use.	TensorFlow has a complex architecture and not easy to use.